Procedurálne programovanie

slido slido.com
#2205705
PrPr 06 (26)

Ján Zelenka

Ústav Informatiky

Slovenská akadémia vied



Obsah prednášky

1. **Opakovanie**
2. **Viacrozmerné polia**
3. **Rekurzívne funkcie**
4. **Konštantné premenné**

Spätná väzba: *https://forms.gle/5ft8RFuLtVb4gRXm8*

Viacrozmerné polia

slido slido.com
#2205705
PrPr 06 (26)

Dvojrozmerné pole

- **statické dvojrozmerné pole**
- pole ukazovateľov
- ukazovateľ na pole
- ukazovateľ na ukazovateľ

Dvojrozmerné pole

Statické dvojrozmerné pole

```
int tabulka[5][4];
```

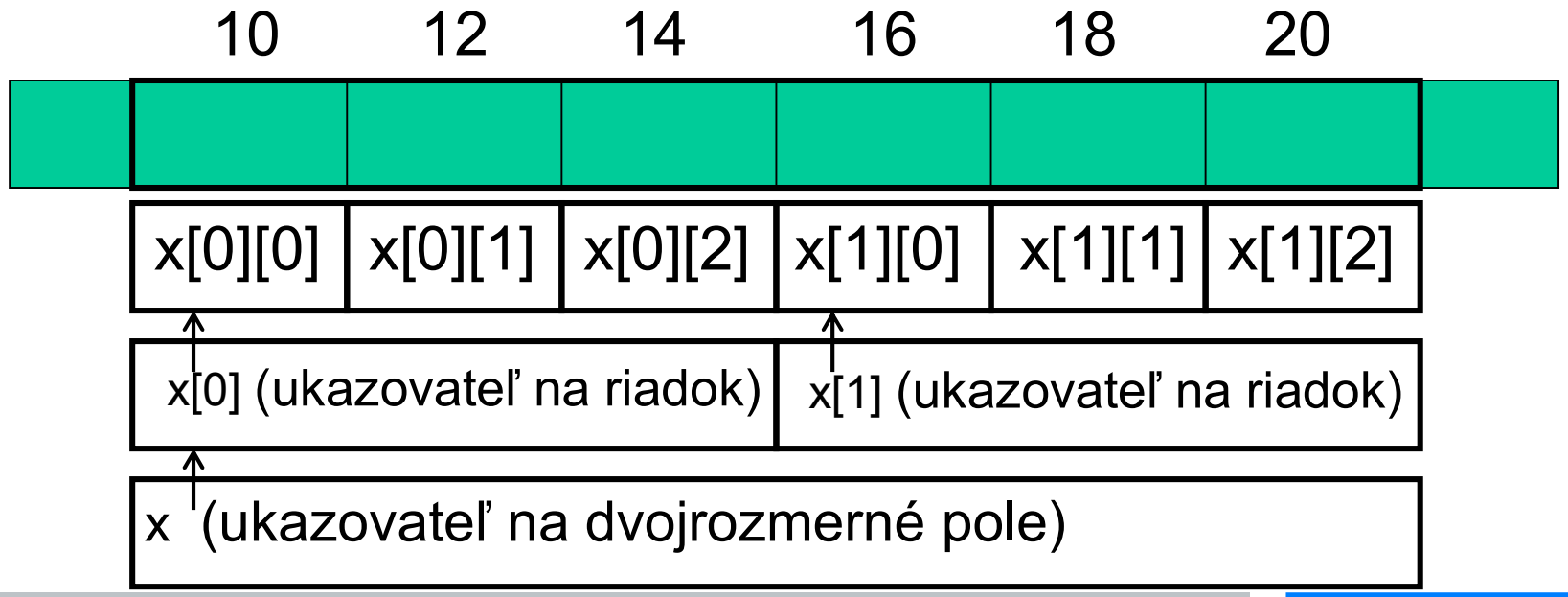
[0][0]	[0][1]	[0][2]	[0][3]
[1][0]	[1][1]	[1][2]	[1][3]
[2][0]	[2][1]	[2][2]	[2][3]
[3][0]	[3][1]	[3][2]	[3][3]
[4][0]	[4][1]	[4][2]	[4][3]

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20

Statické dvojrozmerné pole

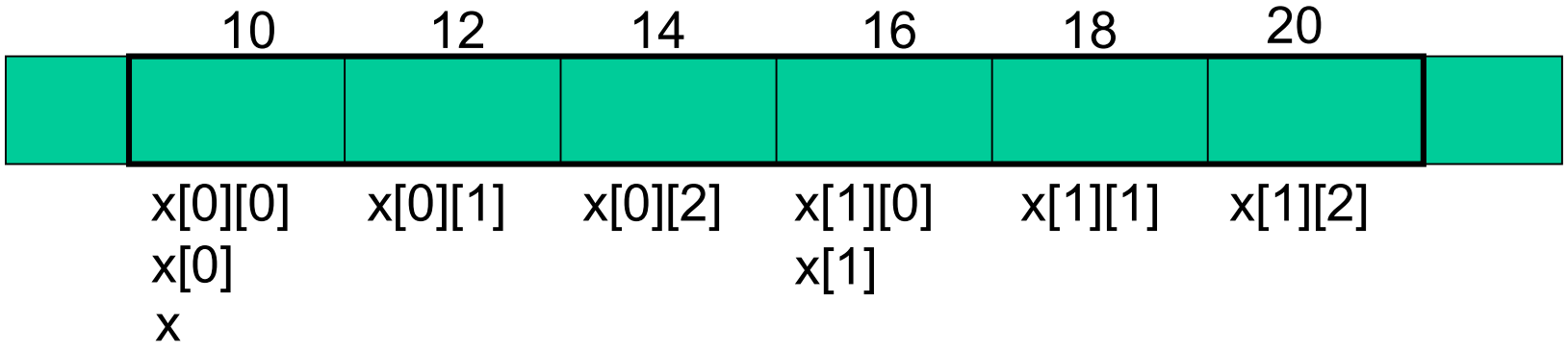
```
int xa[2][3];
```

- pole **xa**:
  - alokované pri preklade
  - súvislý blok 6 prvkov
  - uložené po riadkoch
  - konštantný ukazovateľ



Uloženie viacrozmerného poľa v pamäti

- `x` a `x[0]` - tá istá adresa, len iného typu
- `x+1` a `x[0]+1` - predstavujú odlišné adresy
- `x` - ukazovateľ na dvojrozmerné pole
- `x[i]` - ukazovateľ na i-ty riadok
- `*(x + 1) == x[1] ==16` - adresa prvého riadku
- `x[i][j]` - hodnota prvku dvojrozmerného poľa

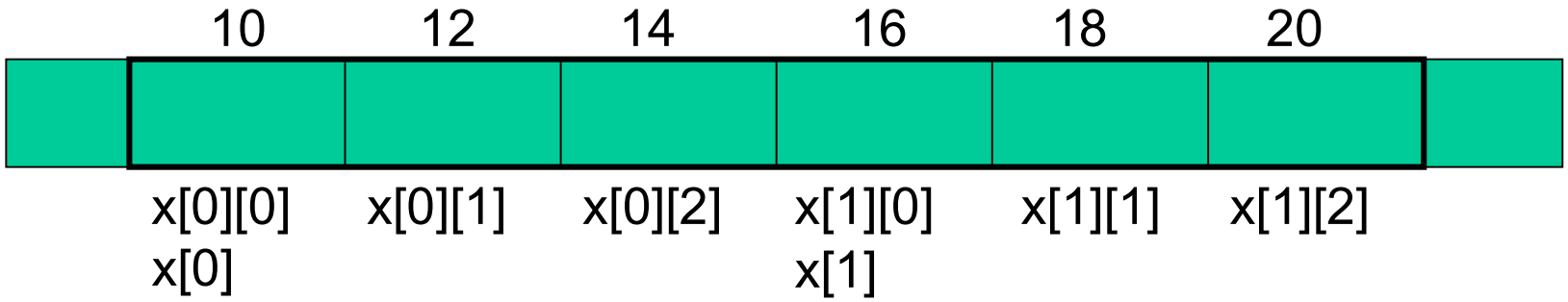


Uloženie viacrozmerného poľa v pamäti

- `x[i] == *(x + i)` - adresa `i`-teho riadku

- `&x[i][j] == x[i] + j == *(x + i) + j`
  - adresa (i,j) pozicie prvku

- `x[i][j] == *(x[i] + j) == *(*(x + i) + j)`
  - hodnota (i,j) pozicie prvku



Dvojrozmerné pole

- statické dvojrozmerné pole
- **pole ukazovateľov**
- ukazovateľ na pole
- ukazovateľ na ukazovateľ

Pole ukazovateľov

```
int *xb[2];
```

- pole **xb**:
  - jednorozmerné pole dvoch ukazovateľov na **int**
  - ukazovatele sa využijú na riadky poľa, pre ktoré musíme alokovať pamäť pre 3 stĺpce

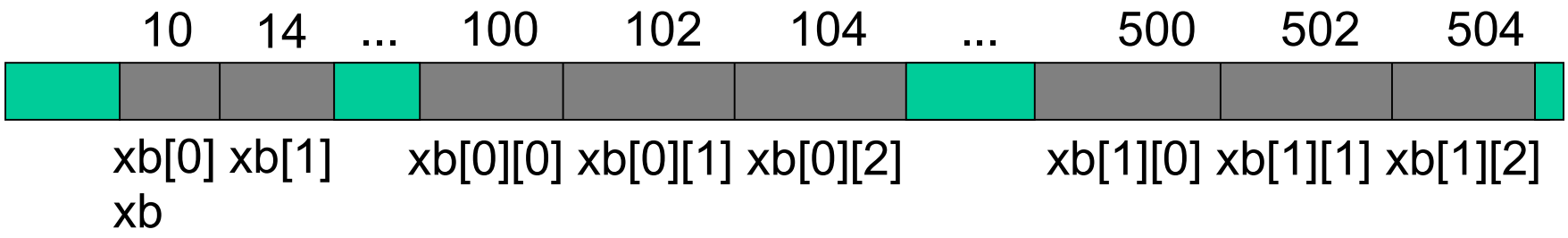
```
xb[0] = (int *) malloc(3* sizeof(int));
xb[1] = (int *) malloc(3* sizeof(int));
```

potom sa dá použiť: `xb[0][2] = 5;`

Pole ukazovateľov

```
int *xb[2];
```

- pole **xb**:
  - jednotlivé riadky nemusia nasledovať v pamäti bezprostredne za sebou
  - ak statickému poľu **int xa[2][3]** priradíme **xa[0][3] = 5**, potom sa hodnota priradí **xa[1][0]**, u poľa **xb** to **nemusí platiť**



Dvojrozmerné pole

- statické dvojrozmerné pole
- pole ukazovateľov
- **ukazovateľ na pole**
- ukazovateľ na ukazovateľ

Ukazovateľ na pole

```
int (*xc)[3];
```

- pole **xc**:
  - **xc** je ukazovateľ na pole troch **int**-ov
  - ak alokujeme dostatok pamäte - ako dvojrozmerné pole

```
xc = (int *) malloc(2 * 3 * sizeof(int));
```

- **xc** ukazuje na pole 6 prvkov združených po troch
- obdoba statického poľa

dá sa použiť: `xc[0][2] = 5;`

Ukazovateľ na pole

```
int (*xc)[3];
```

- pole **`xc`**:
  - jednotlivé riadky nasledujú v pamäti bezprostredne za sebou



Dvojrozmerné pole

- statické dvojrozmerné pole
- pole ukazovateľov
- ukazovateľ na pole
- **ukazovateľ na ukazovateľ**

Ukazovateľ na ukazovateľ

```
int **xd;
```

- pole **xd**:

  (1) alokujeme ukazovatele na riadky

```
xd = (int **) malloc(2 * sizeof(int *));
```

  (2) alokujeme jednotlivé riadky

```
xd[0] = (int *) malloc(3 * sizeof(int));
xd[1] = (int *) malloc(3 * sizeof(int));
```

dá sa použiť:

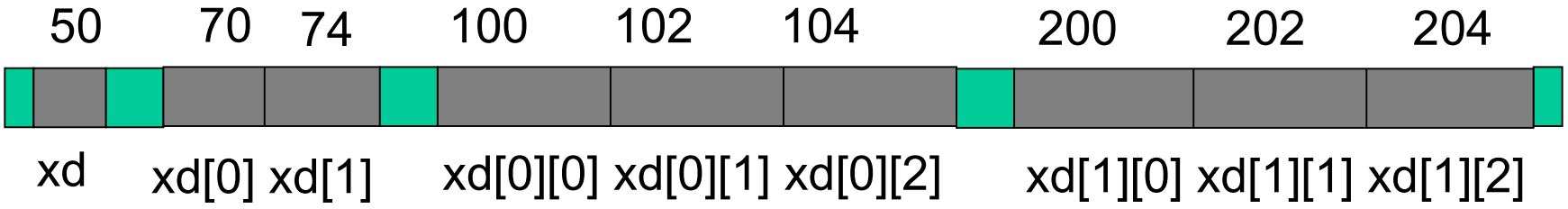
```
xd[0][2] = 5;
```

Ukazovateľ na ukazovateľ

```
int **xd;
```

- pole **xd**:

xd	je ukazovateľ na ukazovateľ na typ **int**
***xd**	je ukazovateľ na typ **int**
****xd**	je prvok typu **int**



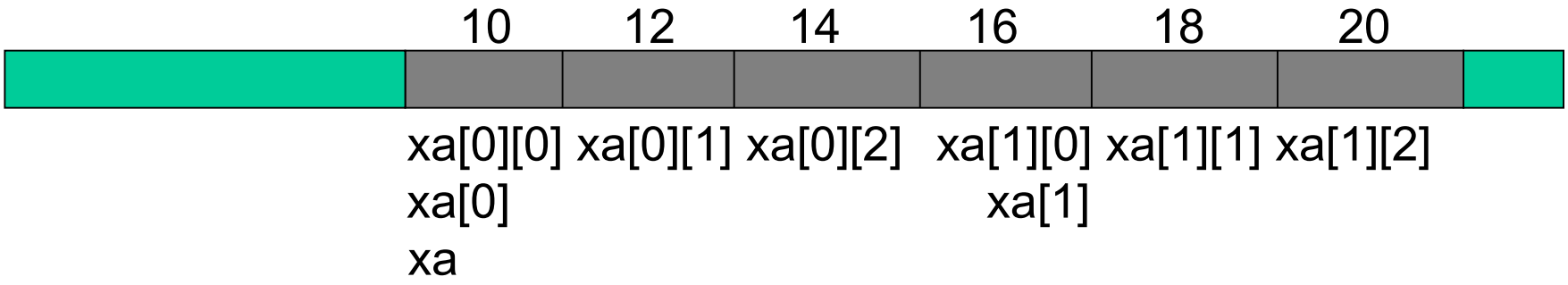
Porovnanie: typ poľa

slido slido.com
#2205705
PrPr 06 (26)

- definícia **`xa`** (**`int xa[2][3]`**) predstavuje statické pole
- definícia **`xb`** (**`int *xb[2]`**), **`xc`** (**`int (*xc)[3]`**) a **`xd`** (**`int **xd`**) predstavujú po alokácii dynamické polia

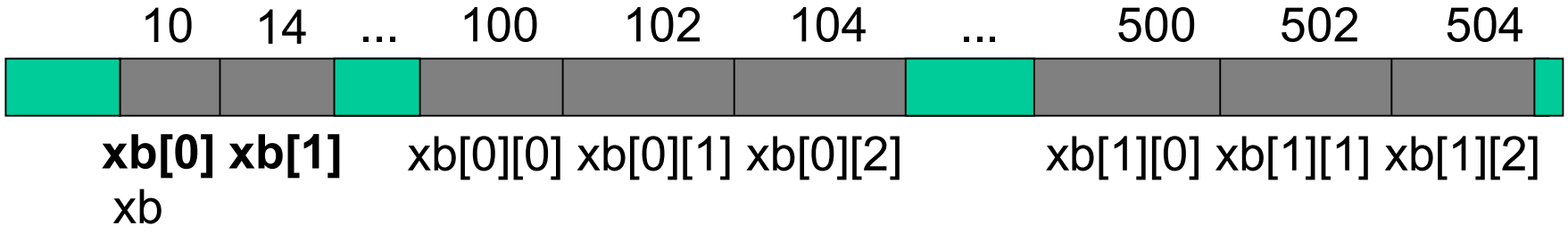
Porovnanie: pamäťové nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšia
  - Výhoda - nie je potrebná alokácia ďalších smerníkov
  - Nevýhoda – potreba veľkého bloku pamäte, problém v prípade poľa veľkého rozsahu



Porovnanie: pamäťové nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšia
- **`xb`** (**`int *xb[2]`**): naviac pamäť pre 2 ukazovatele (počet riadkov **`xb[0]`**, **`xb[1]`**)
  - riadky nie sú v pamäti uložené v celku



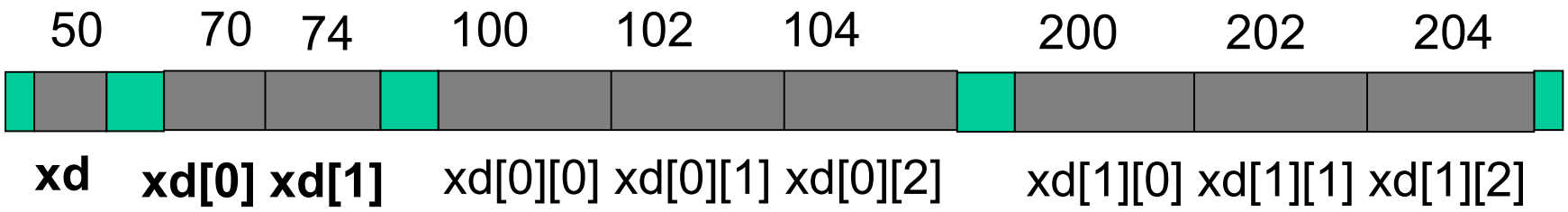
Porovnanie: pamäťové nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšia
- **`xb`** (**`int *xb[2]`**): naviac pamäť pre 2 ukazovatele (počet riadkov **`xb[0]`**, **`xb[1]`**)
- **`xc`** (**`int (*xc)[3]`**): naviac pamäť pre 1 ukazovateľ na typ **`int`**



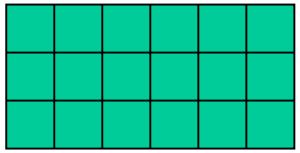
Porovnanie: pamäťové nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšia
- **`xb`** (**`int *xb[2]`**): naviac pamäť pre 2 ukazovatele (počet riadkov **`xb[0]`**, **`xb[1]`**)
- **`xc`** (**`int (*xc)[3]`**): naviac pamäť pre 1 ukazovateľ na typ **`int`**
- **`xd`** (**`int **xd`**): naviac 3 ukazovatele (pre **`xd`** a riadky), najpomalší prístup k prvkom
  - riadky nie sú v pamäti uložené v celku

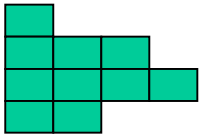


Porovnanie: charakteristika

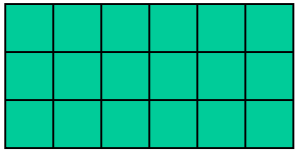
- `xa` (`int xa[2][3]`): pravoúhle pole



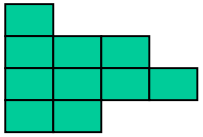
- `xb` (`int *xb[2]`): "zubaté" pole



- `xc` (`int (*xc)[3]`): pravoúhle pole



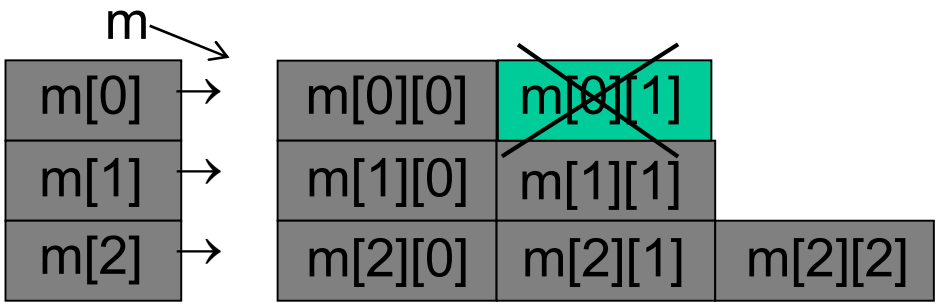
- `xd` (`int **xd`): "zubaté" pole



Zubaté pole

dvojrozmerné pole s rôznou dĺžkou riadkov - časť matice pod diagonálou (vrátane) - **`int *m[3]`**

```
int *m[3], i;
for(i = 0; i < 3; i++)
    m[i] = (int *) malloc((i+1) * sizeof(int));
```



Pravouhlé pole

```
int **alokuj2D(int riadky, int stlpce)
{
  int **p, i;

  p = (int **) malloc(riadky * sizeof(int *));
  for(i = 0; i < riadky; i++)
    p[i] = (int *) malloc(stlpce * sizeof(int));

  return p;
}
```

```
int **a, **b;
a = alokuj2D(3, 5);
b = alokuj2D(10, 20);
```

príklad volania funkcie:

Dvojrozmerné pole ako parameter funkcie

Podobne ako jednorozmerné pole

- odlišnosť:
  - prvá dimenzia - prázdna []
  - druhá dimenzia musí byť uvedená, napr. [10]
- preto
  - je potrebné preniesť do funkcie aj počet riadkov
  - skutočný parameter: len pravouhlé polia (**`xa`**, **`xc`**)

```
double x[5][6];
```

```
double x[][5]
```

alebo

```
double (*x)[5]
```

~~`double *x[5]`~~

Inicializácia polí

```
double f[3] = {1.5, 3.0, 7.6 };
```

```
double f[] = {1.5, 3.0, 7.6 };
```

ak nie je uvedený počet prvkov, určí sa podľa počtu hodnôt.

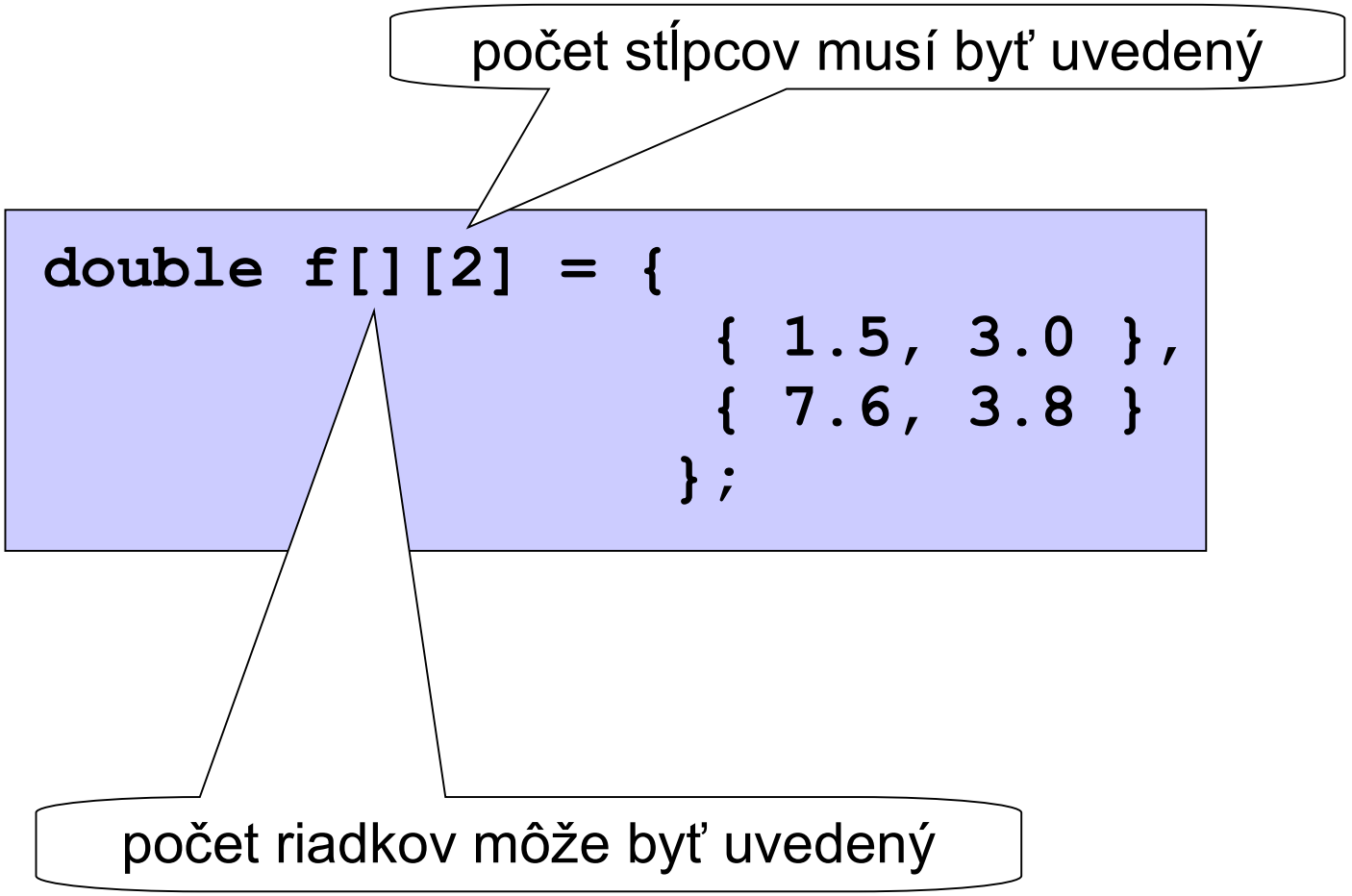
```
double f[3] = {1.5, 3.0 };
```

ak je hodnôt menej, doplní sa hodnotami 0.0

```
double f[3] = {1.5, 3.0, 7.6, 3.8 };
```

ak je hodnôt viac → chyba

Inicializácia dvojrozmerných polí

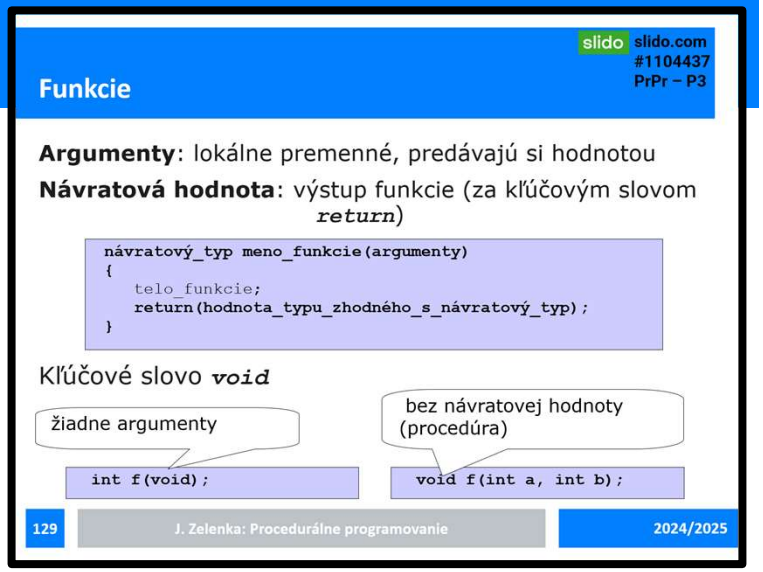


Korektné uvoľnenie pamäte

```
int n;
scanf("%d", &n);
int **pole_2D = malloc(n*sizeof(int*));
for(int i=0; i<n; i++){
    pole_2D[i]= malloc((i+1)*10*sizeof(int));
}
...
for(int i=0;i<n;i++){
    free(pole_2D[i]);
    pole_2D[i] = NULL;
}

free(pole_2D);
pole_2D= NULL;
```

Funkice - rekurzia



Rekurzia

Funkcia, ktorá volá samu seba (väčšinou s inými parametrami).

- rekurzia: viď rekurzia

Niektoré problémy nad vstupnými dátami X sa dajú rozdeliť ako množina tých istých problémov nad menšími dátami.

Rekurzia

- rozdiel nie je syntaktický ale „architektonický“
- rekurzívna funkcia volá vo svojom tele samú seba
  - s upravenými argumentami
  - vytvorí sa samostatný rámec na zásobníku
    - kópia lokálnych premenných pre každé vnorenie (rekurzívne zavolanie funkcie)
  - po skončení vnoreného volania, pokračuje sa v aktuálnom
- vnáranie sa zastavuje na triviálnom prípade
  - volenie funkcie, ktoré nespôsobí opätovné vnorenie
  - zlá charakterizácia vedie k nekonečnej rekurzii

Fibonacciho čísla

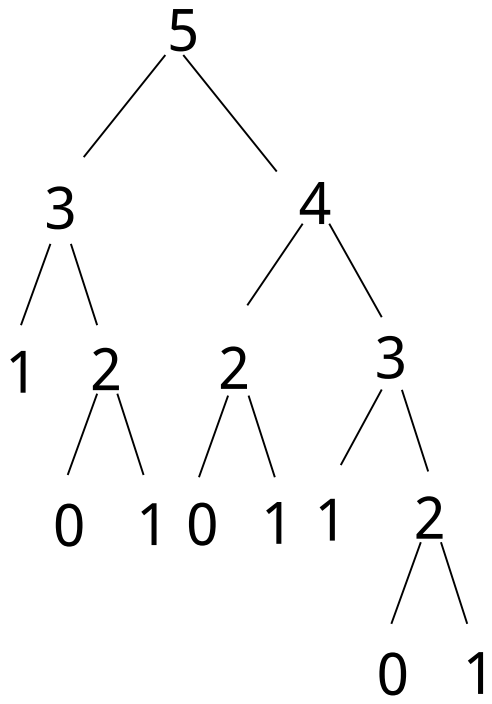
0, 1,1, 2, 3, 5, 8, 13, 21, ...

```
f(1) = 1
f(2) = 1
f(n) = f(n-1) + f(n-2)
```

- čísla v postupnosti sú súčtom dvoch predchádzajúcich

Fibonacciho čísla
Rekurzívne


```
int fib(int n)
{
   if (n <= 1)
      return n;
   else
      return fib(n-2) + fib(n-1);
}
```



neefektívne, pretože sa veľakrát vypočítavajú tie isté čísla - iteratívne s použitím poľa - efektívne

Fibonacciho čísla
Iteratívne

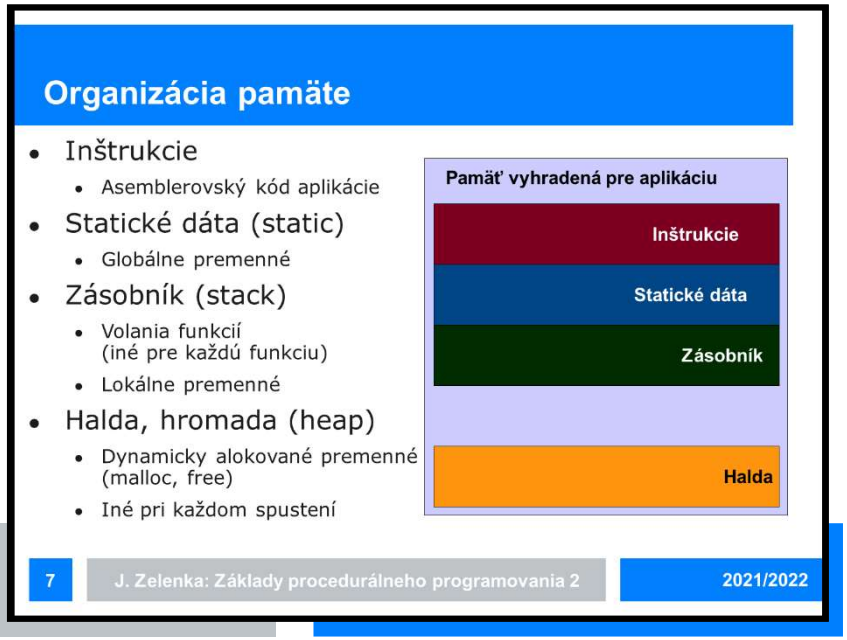

```
int fib(int n)
{
   int n_1, n_2, i, sucet;
   if (n <= 1)
      return n;
   else {
      n_1 = 0;
      n_2 = 1;

      for(i=2; i<=n; i++) {
         sucet = n_1 + n_2;
         n_1 = n_2;
         n_2 = sucet;
      }
      return sucet;
   }
}
```

iteratívne:
nerekruzívne, s
použitím cyklov

Rekurzívne funkcie

- zavolanie funkcie vyžaduje nadbytočnú réžiu (pamäť CPU)
  - posielanie argumentov, návratovej hodnoty, práca so zásobníkom…
  - najmä pri početných vnoreniach
- každá rekurzívna funkcia sa dá vyjadriť v nerekurzívnom tvare

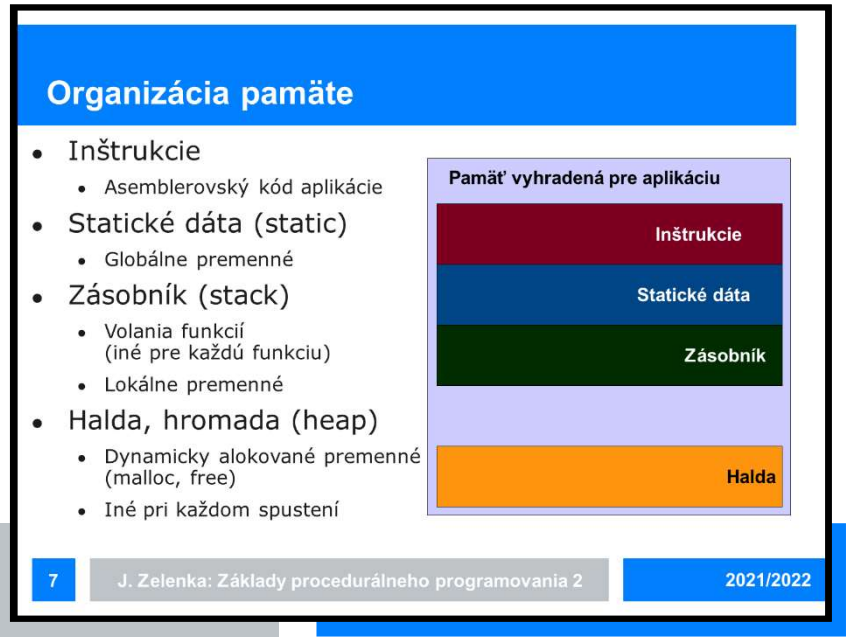


Debugovanie rekurzívnych funkcií

- sledovanie zásobníka (call stack)
- podmienené breakpointy
  - na záujmovú vstupnú hodnotu

Vymedzenie pamäte v zásobníku

- zaisťuje kompilátor pri volaní funkcie
- väčšina lokálnych premenných definovaných vo funkciách
- existencia týchto premenných začína pri vstupe do funkcie a **končí pri výstupe** z funkcie



Naozaj funkcia nikdy neskončí?

slido slido.com #2205705 PrPr 06 (26)

```
void nekonecno(){
  printf("Nikdy neskoncim!?\n");
  nekonecno();
}

int main(void)
{
    nekonecno();
    return 0;
}
```

Každé ďalšie zavolanie funkcie zvýši hĺbku vnorenia a vyžaduje pamäť na zásobníku volaní. Zásobník ma obmedzenú veľkosť (závislú od kompilátora alebo od prostredia operačného systému), a preto v konečnom čase (zvyčajne do par sekúnd) tento program skonči.

Porovnanie iteratívne VS rekurzia

Iteratívne riešenie

```
int factorial(int n) {
    int i, result = 1;
    for (i = 1; i <= n; i++) {
        result = result * i;
    }
    return result;
}
```

Rekurzia

```
int factorial(int n) {
    if (n < 2)
      return 1;
    else
      return n * factorial(n-1);
}
```

Faktoriál prirodzeného čísla *n*

- *n!* -> súčin všetkých celých čísel od 1 po *n*.
- 0! definujeme ako 1.

Porovnanie iteratívne VS rekurzia

Iteratívne riešenie

```
int gcd(int a, int b) {
    while(b != 0) {
        int x = a % b;
        a = b;
        b = x;
    }
    return a;
}
```

Euklidov algoritmus – výpočet najväčšieho spoločného deliteľa

Rekurzia

```
int gcd(int a, int b) {
   if (b == 0)
      return a;
   else
      return gcd(b, a % b);
}
```

Porovnanie iteratívne VS rekurzia

Iteratívne riešenie

```
int fib(int n)
{
   int n_1, n_2, i, sucet;
   if (n <= 1)
      return n;
   else {
      n_1 = 0;
      n_2 = 1;
      for(i=2; i<=n; i++) {
         sucet = n_1 + n_2;
         n_1 = n_2;
         n_2 = sucet;
      }
      return sucet;
   }
}
```

Rekurzia

```
int fib(int n)
{
   if (n <= 1)
      return n;
   else
      return (fib(n-2) +
              fib(n-1));
}
```

Fibonačiho čísla

Konštantné premenné

slido slido.com
#2205705
PrPr 06 (26)

Pozor, **konštantné výrazy** (napr. definícia statického poľa) sa definujú **pomocou direktívy predprocesora**
(ukážeme si na nasledujúcej prednáške)

Kľúčové slovo const

- označuje nemennú (*read-only*) **premennú**
  - napr. matematické konštanty, stavy programu…
- prechádzame nechceným implementačným chybám
  - jej hodnota sa nemení
  - argument funkcie, ktorý sa nesmie zmeniť

```
void procedura(const float f){
    const int a;
    cont int i = 10;
}
```

chyba, neinicializovaná konštanta

Kľúčové slovo const

- používajte ho čo najčastejšie
  - zlepšuje typovú kontrolu
  - nemeníte konštantné premenné
  - lepšia optimalizácia prekladačom
  - zlepšuje prácu ďalším programátorom s vaším kódom
- premenná s **const** je lokálna v danom súbore

Reťazcové konštanty

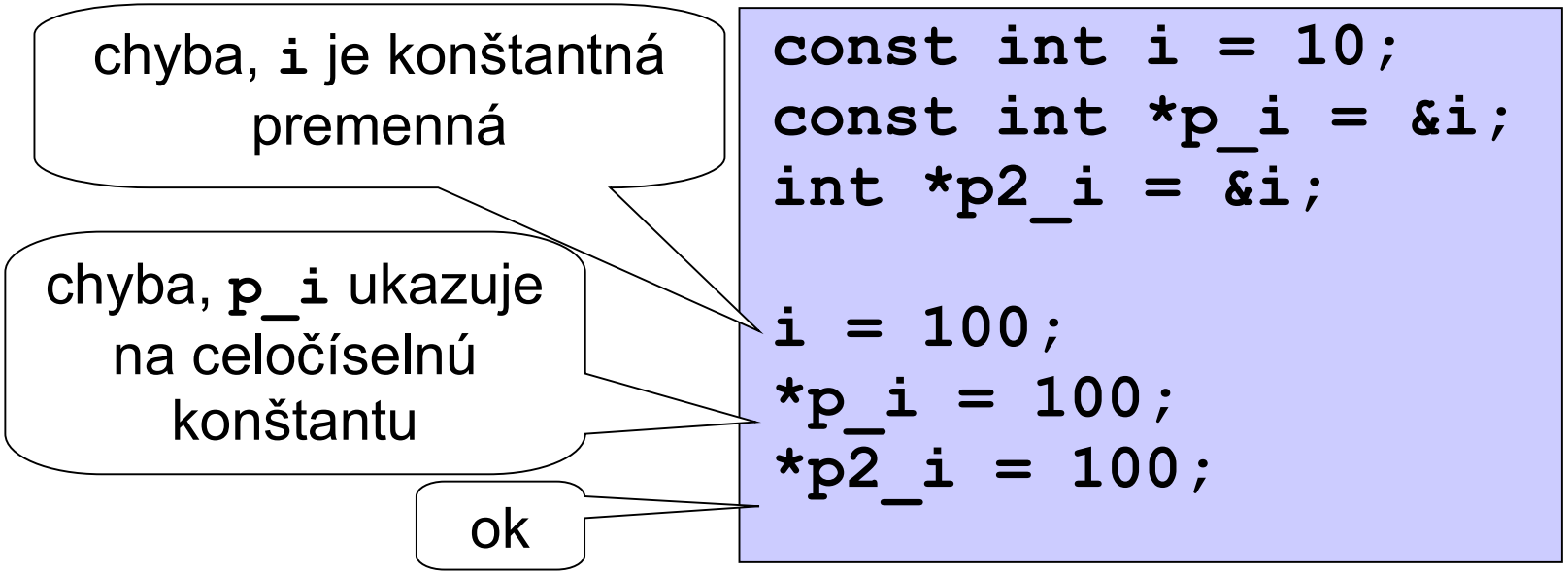
"Ahoj"

- uložený v statickej časti
- je typu **`char*`**
- Vhodnejšie je s ním pracovať ako **`const char*`**

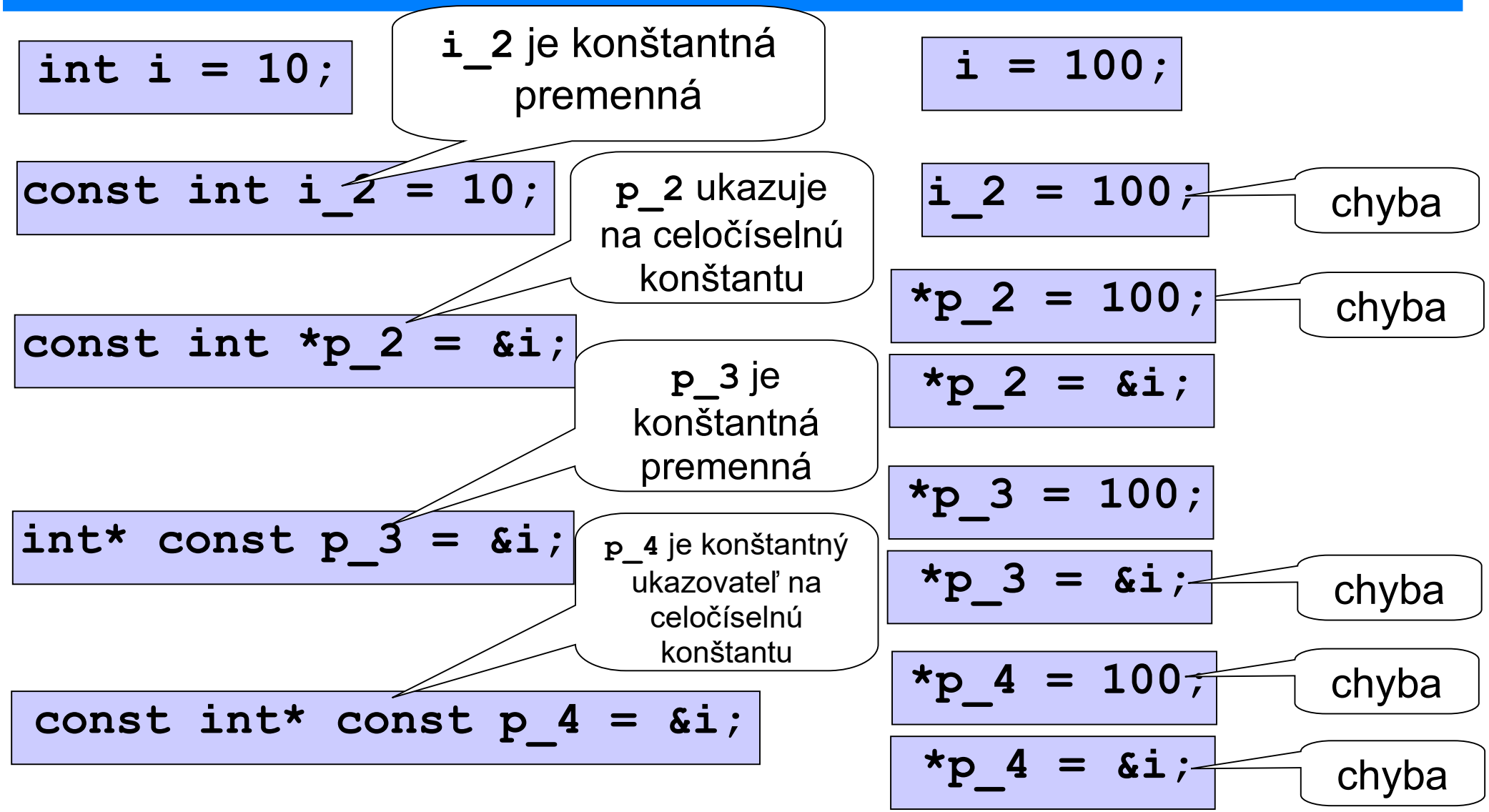
```
const char *p_text[4];
```

Kľúčové slovo const a ukazovateľ

- konštantná je iba hodnota premennej
  - platí pre ukazovateľ a jeho dereferencovanie
- Nie je konštantné miesto, kam ukazovateľ ukazuje
  - const je plytké
  - konštantnú premennú vieme modifikovať cez nekonštantný ukazovateľ



Kľúčové slovo const a ukazovateľ



Ukazovateľ na konštantu argumentom funkcie


```
void procedura(const int *pole){
    pole[0] = 10;
    int *pom_pole = pole;
    pom_pole[0] = 10;
}
```

chyba, pole je "read-only"

ok

warning: initialization discards '**const**' qualifier from pointer target type

Ďakujem vám za pozornosť!

slido.com
#2205705
PrPr 06 (26)

Spätná väzba: *https://forms.gle/5ft8RFuLtVb4gRXm8*